> *From: https://chatgpt.com/c/6a8a2356-f96c-83ee-950e-82f71a5e00eb*

you asked: go throgh

message time: 2026-08-23 04:00:40

go throgh the previous chats from start to end completly which are attached files and responces how detailed- https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93 and https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5 also - https://chatgpt.com/c/6a8a20f0-f3fc-83ee-891b-ba811975393f

And continue further according to previous - Module 1,2,3 completed as aboved thier links given and also attached files . Now continue with **Module 4:** 75 REST/API questions.

For your goal, I recommend turning this into a structured **"2-Year Java + Spring Boot Product Company Interview Bible"** with:

Module 1: 150 Core Java questions + answers

Module 2: 120 Spring/Spring Boot questions + internal working

Module 3: 80 JPA/Hibernate questions

Module 4: 75 REST/API questions

Module 5: 75 Microservices questions

Module 6: 50 Spring Security/JWT questions

Module 7: 50 SQL questions + queries

Module 8: 50 Multithreading questions

Module 9: 50 Kafka + Redis questions

Module 10: 50 Docker/AWS questions

Module 11: 100 DSA problems specifically for 2-year Java SDE interviews

Module 12: 50 LLD/system-design scenarios

Module 13: 100 project/resume follow-up questions

Module 14: Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions**.

chatgpt response: Yes. I wen

Yes. I went through the **three attached conversation exports** and used them as the continuity source for the interview bible.

The previous material establishes that:

- The overall structure is **14 modules**, with Module 4 = **75 REST/API questions**.
- The intended learning format is **Question → Short interview answer → Detailed explanation → Real project example → Follow-up → Tricky counter-question → Best answer → Code/diagram where relevant**.
- Module 2 already covered the **Spring MVC request-processing foundation**, including `DispatcherServlet`, `HandlerMapping`, controllers, DTOs, validation, exception handling, HTTP status codes, idempotency, pagination and API versioning.
- Module 3 was deliberately connected back to the REST layer, e.g. REST API → Service → `@Transactional` → Repository → JPA → Hibernate → SQL → DB.

So **Module 4 should not simply repeat Module 2's Spring annotations**. Instead, we'll now go deeper into **REST/API design itself**: HTTP semantics, resource modeling, API contracts, errors, validation, pagination, filtering, sorting, versioning, caching, concurrency, idempotency, security, rate limiting, retries, observability, performance and real production scenarios.

Module 4 — REST / API

75 Interview Questions + Answers

For 2-Year Java + Spring Boot Product-Company Interviews

Module 4 roadmap

Part	Questions	Focus
A	Q1–15	REST fundamentals + HTTP
B	Q16–30	API design + Spring REST
C	Q31–45	Validation + errors + pagination
D	Q46–60	Idempotency + caching + concurrency + security
E	Q61–75	Production scenarios + performance + tricky interviews

The key mindset remains the same: **don't memorize an annotation; understand the complete request/response and design decision.** The previous Spring material explicitly emphasized following an HTTP request all the way through Tomcat → `DispatcherServlet` → controller → service → repository → database → response.

PART A — REST FUNDAMENTALS + HTTP

Q1–Q15

Q1. What is REST? □□

Short interview answer

REST stands for **Representational State Transfer**. It is an architectural style for designing distributed systems where resources are identified by URLs and manipulated using standard HTTP methods.

Example:

```
text
 GET    /api/users/10
 POST   /api/users
 PUT    /api/users/10
 PATCH  /api/users/10
 DELETE /api/users/10
```

Detailed explanation

REST is not a framework.

It is a set of architectural constraints/principles.

Important REST concepts include:

```
text
 Resource
    ↓
 URI
    ↓
 HTTP method
    ↓
 Representation
    ↓
 HTTP response
```

For example:

```
text
 Resource = User
 URI      = /api/users/10
```

```
Method   = GET
Response = JSON representation
```

A REST API normally uses HTTP as its application protocol.

Important REST constraints

You should know:

1. Client-server separation
2. Statelessness
3. Cacheability
4. Uniform interface
5. Layered system
6. Code-on-demand — optional

Real project example

For an e-commerce application:

```
text
 GET /api/orders/1001
```

returns the representation of order `1001`.

Follow-up

Is REST the same as HTTP?

No.

HTTP is a protocol.

REST is an architectural style that can be implemented using HTTP.

Tricky interviewer counter-question

Is every HTTP API RESTful?

No.

An API can use HTTP but violate important REST principles.

Best answer

> *REST is an architectural style, while HTTP is a communication protocol. Most modern REST APIs use HTTP because HTTP provides methods, status codes, headers, caching and other mechanisms that fit REST very well.*

Q2. What are the main principles of REST? ⭐

Short answer

The major REST principles are:

```
text
 Client-Server
 Stateless
 Cacheable
 Uniform Interface
 Layered System
 Code-on-Demand (optional)
```

Detailed explanation

1. Client-server

Client and server have separate responsibilities.

```
text
 Frontend
    ↓ HTTP
 Backend API
    ↓
 Database
```

The client doesn't need to know how the server stores data.

2. Stateless

The server should not depend on conversational client state stored between requests.

Every request should contain the information needed to process it.

For example:

```
http
 GET /api/orders/100


 Authorization: Bearer <token>
```

The server should not need to remember the previous request.

3. Cacheable

Responses should indicate whether they can be cached.

HTTP provides mechanisms such as:

```
http
 Cache-Control
 ETag
 Last-Modified
 Expires
```

4. Uniform interface

Resources should be accessed consistently.

```
text
 GET    /users
 GET    /users/10
 POST   /users
 PUT    /users/10
 DELETE /users/10
```

5. Layered system

The client doesn't necessarily know whether it is communicating directly with the application.

There may be:

```
text
 Client
  ↓
 Load Balancer
  ↓
 API Gateway
  ↓
 Application
  ↓
 Database
```

Interview trap

Does REST require JSON?

No.

REST does not require JSON.

JSON is simply a very common representation format.

Q3. What is a resource in REST? □

Short answer

A resource is a conceptual entity exposed through an API and identified by a URI.

Examples:

```text
 /users
 /orders
 /products
 /payments
```

Example

```http
 GET /api/users/101
```

Here:

```text
 /users/101
```

identifies the user resource.

Important distinction

Resource:

```text
 User
```

Representation:

```json
 {
   "id": 101,
   "name": "Rahul"
 }
```

The resource is the conceptual object.

JSON is one representation of that resource.

Tricky question

Is `/users/101` the resource?

Strictly speaking, the URI identifies the resource.

The resource itself is the conceptual entity.

Q4. How should REST URLs be designed? □

Good design

Use **nouns**, not verbs.

Prefer:

```
text
 GET /users/10
```

rather than:

```
text
 GET /getUser/10
```

Prefer:

```
text
 POST /orders
```

rather than:

```
text
 POST /createOrder
```

Common pattern

```
text
 /users
 /users/{id}

 /orders
 /orders/{id}

 /products
 /products/{id}
```

Nested resources

For relationships:

```
text
 GET /users/10/orders
```

can represent orders belonging to user `10`.

Bad examples

```
text
 /getUsers
 /createUser
 /deleteUser
 /updateUser
```

The HTTP method already communicates the operation.

Follow-up

Should URLs contain database table names?

Not necessarily.

The API contract should model the domain, not expose internal database implementation details.

Q5. Explain HTTP methods used in REST. □□

Method	**Typical purpose**
GET	Read
POST	Create/process
PUT	Replace
PATCH	Partial modification
DELETE	Delete

Example

```
text
 GET    /users/10
 POST   /users
 PUT    /users/10
 PATCH  /users/10
 DELETE /users/10
```

Important

Don't simply say:

> *POST = insert, PUT = update.*

That's too simplistic.

HTTP methods describe semantics, not necessarily direct database operations.

Q6. PUT vs PATCH? □□

Short answer

PUT generally replaces the resource representation.

PATCH performs a partial modification.

Example:

Current resource:

```
json
 {
   "name": "Rahul",
   "email": "rahul@gmail.com",
   "age": 25
 }
```

PUT:

```
http
 PUT /users/10
```

could send the complete desired representation.

PATCH:

```
http
 PATCH /users/10
```

could send:

```
json
 {
   "email": "new@gmail.com"
 }
```

Interview trap

Is PUT always idempotent?

PUT is defined as idempotent by HTTP semantics when repeated requests have the same intended effect.

PATCH can be idempotent or non-idempotent depending on the operation.

Q7. POST vs PUT? □

POST

Usually used to create a subordinate resource where the server determines the identifier.

```http
POST /users
```

Server:

```text
201 Created
Location: /users/101
```

PUT

Often used when the client knows the resource URI.

```http
PUT /users/101
```

Tricky question

Can POST be idempotent?

Yes, application design can make POST effectively idempotent, for example by using an idempotency key.

But POST itself is not inherently idempotent.

Q8. What does idempotency mean? □□□

Short interview answer

An operation is idempotent when performing the same request multiple times has the same intended effect as performing it once.

The previous Spring material already identified idempotency as a key API concept because it becomes particularly important when designing retries in distributed systems.

Example

```http
PUT /users/10
```

with:

```json
{
  "name": "Rahul"
}
```

Repeatedly sending the same request should leave the user in the same state.

Important distinction

Idempotent does **not** necessarily mean:

> *The response must be identical.*

It means the intended state-changing effect is the same.

Real-world example

Payment APIs are a classic case.

Suppose:

```text
Client → Payment API
```

Network timeout occurs.

Client doesn't know whether payment succeeded.

It retries.

Without idempotency:

```text
₹100
↓
Payment 1


₹100
```

```
↓
Payment 2
```

With:

```http
Idempotency-Key: abc123
```

the server can recognize the retry.

Q9. Which HTTP methods are idempotent? □

Generally:

```text
GET      yes
HEAD     yes
PUT      yes
DELETE   yes
OPTIONS  yes
TRACE    yes
POST     no
PATCH    not inherently
```

Important interviewer nuance

Idempotency is about **semantics**, not whether the implementation happens to produce the same result.

For example:

```http
DELETE /users/10
```

First request:

```text
204
```

Second request:

```text
404
```

The operation can still be considered idempotent because the intended state—user `10` no longer exists—is unchanged.

Q10. What is statelessness in REST? □

Short answer

Each request should contain the information required to process it, and the server should not depend on previous client requests to understand the current request.

Example

```http
GET /orders/100


Authorization: Bearer token
```

The server validates the token for that request.

It doesn't depend on:

```text
previousRequest = login()
```

being stored as conversational state.

Important distinction

Stateless doesn't mean:

> *Server cannot store anything.*

The server can store:

```text
users
orders
sessions
database records
cache
```

Statelessness refers to **client interaction state required to process requests**.

Q11. What is the difference between stateful and stateless APIs?

Stateful

Server maintains client interaction state.

```text
 Client
  ↓
 Server
  ↓
 Session
```

Subsequent requests depend on that session.

Stateless

Each request contains what is needed.

```text
 Request 1 → independent
 Request 2 → independent
 Request 3 → independent
```

Why stateless APIs are useful

They make horizontal scaling easier:

```text
                ┌── Server 1
 Client → LB ──┼── Server 2
                └── Server 3
```

Any server can process the request.

Tricky question

Does JWT automatically make an API stateless?

No.

JWT can help with stateless authentication, but if your application still relies on server-side session state, the overall architecture may still be stateful.

Q12. What is the difference between safe and idempotent HTTP methods?

This is a favorite follow-up.

Safe

A safe method is intended for read-only semantics.

Examples:

```
text
 GET
 HEAD
 OPTIONS
```

Idempotent

Repeated requests have the same intended state-changing effect.

Examples include:

```
text
 GET
 PUT
 DELETE
```

Therefore

```
text
 GET → safe + idempotent
 PUT → idempotent but not safe
 DELETE → idempotent but not safe
 POST → generally neither
```

Interview trap

Does safe mean the server performs absolutely no side effects?

Not necessarily.

A GET should not change the resource state, but infrastructure-level side effects such as logging or metrics can still occur.

Q13. What are HTTP status codes and why are they important? □

They communicate the outcome of an HTTP request.

Categories

```
text
 1xx → informational
 2xx → success
 3xx → redirection
```

```
4xx → client-side error
5xx → server-side error
```

Important REST codes

```
text
200 OK
201 Created
202 Accepted
204 No Content

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
405 Method Not Allowed
409 Conflict
415 Unsupported Media Type
422 Unprocessable Content
429 Too Many Requests

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
```

The earlier interview material specifically highlighted `200`, `201`, `204`, `400`, `401`, `403`, `404`, `409`, `422`, `500`, and `503`.

Q14. 401 vs 403? □□□

401 Unauthorized

Usually means:

> *Authentication is missing or invalid.*

Example:

```
text
No token
```

```
Expired token
Invalid token
```

403 Forbidden

Authentication may be valid, but the authenticated user doesn't have permission.

```text
User authenticated
        ↓
Role = USER
        ↓
Admin endpoint
        ↓
403 Forbidden
```

Interview answer

> *401 is primarily an authentication problem; 403 means the request is understood and the caller is authenticated or otherwise identified, but access is not permitted.*

Q15. 400 vs 422? ⭐

400 Bad Request

The request is malformed or cannot be processed as a valid request.

Example:

```json
{
  "age": "abc"
}
```

when the API expects a number.

422

The request syntax may be valid, but semantic validation fails.

Example:

```json
{
```

```
  "age": -5
}
```

The JSON is valid, but the value violates the domain rules.

Interview nuance

There isn't one universally enforced convention for every API. The important thing is to establish a consistent API error contract.

PART B — API DESIGN + SPRING REST

Q16–Q30

The previous Spring module already established the internal request chain:

```text
Client
  ↓
Tomcat
  ↓
DispatcherServlet
  ↓
HandlerMapping
  ↓
HandlerAdapter
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

and response conversion through `HttpMessageConverter`.

Now we move beyond that foundation.

Q16. Explain the complete REST request flow in Spring Boot. □□□

Answer

```text
Client
 ↓
HTTP Request
 ↓
Tomcat
 ↓
Servlet Filter
 ↓
DispatcherServlet
 ↓
HandlerMapping
 ↓
HandlerAdapter
 ↓
Controller
 ↓
Service
 ↓
Repository
 ↓
Database
 ↓
Repository
 ↓
Service
 ↓
Controller
 ↓
HttpMessageConverter
 ↓
JSON
 ↓
HTTP Response
 ↓
Client
```

Example

```http
GET /api/users/10
```

Spring identifies the controller method:

```java
@GetMapping("/users/{id}")
public UserResponse getUser(@PathVariable Long id) {
    return userService.getUser(id);
}
```

Important internal components

You should know:

- `DispatcherServlet`
- `HandlerMapping`
- `HandlerAdapter`
- argument resolvers
- `HttpMessageConverter`
- exception resolvers

This is deeper than simply knowing `@GetMapping`.

Q17. `@RequestParam` vs `@PathVariable` vs `@RequestBody`? □□

`@PathVariable`

Identifies a resource.

```http
GET /users/10
```

```java
@GetMapping("/users/{id}")
public User get(@PathVariable Long id)
```

`@RequestParam`

Represents query parameters.

```http
GET /users?page=2&size=20
```

```java
 @RequestParam int page
```

`@RequestBody`

Represents request payload.

```http
 POST /users
 Content-Type: application/json
```

```java
 @RequestBody CreateUserRequest request
```

The previous module established exactly this distinction.

Q18. What is content negotiation?

Short answer

Content negotiation allows the client and server to agree on the representation format.

For example:

```http
 Accept: application/json
```

The client is saying:

> *I prefer JSON.*

The server may return:

```http
 Content-Type: application/json
```

Other examples

```text
 application/json
 application/xml
 text/plain
```

Important headers

```text
 Accept
 Content-Type
```

Difference

`Accept`:

> *What response format do I want?*

`Content-Type`:

> *What format is this request/response body?*

Q19. What is `Content-Type`? ⭐

It describes the media type of the HTTP body.

Example:

```http
 Content-Type: application/json
```

means:

> *The request body contains JSON.*

Example:

```json
 {
   "name": "Rahul"
 }
```

If the server doesn't support the requested representation, it may return:

```text
 415 Unsupported Media Type
```

Q20. What is `Accept` header?

It tells the server which response media types the client can accept.

Example:

```http
Accept: application/json
```

Possible response:

```http
Content-Type: application/json
```

Follow-up

Can `Accept` and `Content-Type` be different?

Yes.

For example:

```text
Request:
Content-Type: application/json

Response:
Content-Type: application/xml
```

if the API supports that and the client's `Accept` permits it.

Q21. Why should we use DTOs instead of exposing entities directly? □□□

Short answer

DTOs separate the external API contract from the persistence model.

Bad approach

```java
@GetMapping("/{id}")
public UserEntity getUser(...) {
    return userRepository.findById(...);
}
```

This couples:

```text
API
 ↓
JPA Entity
```

```
  ↓
 Database model
```

Better

```
text
 Controller
  ↓
 UserResponse DTO
  ↓
 Service
  ↓
 User Entity
```

Benefits

1. API contract independence
2. Prevent exposing internal fields
3. Control JSON structure
4. Validation separation
5. Prevent accidental entity mutation
6. Better versioning
7. Avoid serialization problems

Real example

Entity:

```
java
 User {
     id
     name
     passwordHash
     createdAt
 }
```

Response:

```
json
 {
   "id": 10,
```

```
  "name": "Rahul"
}
```

You absolutely don't want:

```
text
passwordHash
```

in the API response.

Q22. Entity vs DTO?

Entity	**DTO**
Persistence model	API/data-transfer model
Managed by JPA	Usually not
Represents DB/domain persistence	Represents API contract
Can contain relationships	Usually controlled structure
Internal	External boundary

Interview answer

> *I generally avoid exposing JPA entities directly from REST controllers because it couples the API contract to persistence and can expose internal fields or relationships. DTOs give us explicit control over request and response contracts.*

Q23. What is API contract?

An API contract defines how clients interact with the API.

It includes:

```
text
URL
HTTP method
Request headers
Request body
Response body
Status codes
Error format
Authentication
```

```
 Pagination
 Validation rules
```

Example:

```
text
 POST /api/v1/users
```

Request:

```
json
 {
   "name": "Rahul",
   "email": "rahul@gmail.com"
 }
```

Response:

```
text
 201 Created

json
 {
   "id": 101,
   "name": "Rahul"
 }
```

Why important?

Once external clients depend on the contract, changing it can break them.

Q24. What is backward compatibility in APIs? □

An API change is backward-compatible if existing clients continue working.

Suppose V1 returns:

```
json
 {
   "name": "John",
   "age": 30
 }
```

If you suddenly remove:

```text
 age
```

existing clients may break.

Safer evolution

Add fields rather than remove/change existing ones.

```json
 {
   "name": "John",
   "age": 30,
   "email": "john@example.com"
 }
```

Breaking changes

Examples:

```text
 Remove field
 Rename field
 Change data type
 Change semantics
 Change required field
 Change response structure
```

Q25. What is API versioning? □□

API versioning allows an API to evolve while maintaining compatibility for existing clients.

The previous Spring module already introduced:

```text
 /api/v1/users
 /api/v2/users
```

as a common approach.

Common strategies

1. URI versioning

```text
 /api/v1/users
 /api/v2/users
```

2. Query parameter

```text
 /api/users?version=2
```

3. Header

```http
 X-API-Version: 2
```

4. Media type

```http
 Accept: application/vnd.company.user-v2+json
```

Product-company answer

> *I prefer a consistent versioning strategy based on client needs. URI versioning is easy to understand and operate, while media-type/header versioning keeps resource URLs stable.*

Q26. Should every API be versioned?

No.

Versioning introduces maintenance cost.

You should first distinguish:

```text
 Non-breaking evolution
         vs
 Breaking change
```

If you can safely add fields while preserving compatibility, a new version may not be necessary.

Tricky question

What would you do if a response needs a completely different structure?

That is a stronger candidate for a new API version or a new resource representation.

Q27. What is `ResponseEntity`? ⭐

`ResponseEntity` allows the controller to explicitly control:

```
text
 status
 headers
 body
```

Example:

```
java
 return ResponseEntity
         .status(HttpStatus.CREATED)
         .body(response);
```

Or:

```
java
 return ResponseEntity
         .noContent()
         .build();
```

Why useful?

Because APIs often need different HTTP statuses depending on business outcomes.

Q28. `ResponseEntity` vs simply returning an object?

Returning:

```
java
 public UserResponse getUser()
```

lets Spring handle the response.

Using:

```
java
 ResponseEntity<UserResponse>
```

gives explicit control.

Example:

```
java
 return ResponseEntity
```

```
        .status(HttpStatus.CREATED)
        .header("Location", "/users/101")
        .body(response);
```

Interview answer

> *I use direct return types when the response behavior is simple, and ResponseEntity when I need explicit control over status, headers, or body.*

Q29. What is `Location` header for `201 Created`? ⭐

When creating a resource, the server can return:

```http
HTTP/1.1 201 Created
Location: /api/users/101
```

This tells the client where the newly created resource can be accessed.

Example:

```http
POST /api/users
```

Response:

```http
201 Created
Location: /api/users/101
```

Strong answer

> *For resource creation, 201 Created indicates successful creation, and the Location header can identify the URI of the newly created resource.*

Q30. What is HATEOAS?

HATEOAS stands for:

Hypermedia As The Engine Of Application State.

The API response contains links describing available actions/resources.

Example:

```json
{
  "id": 101,
  "name": "Rahul",
  "_links": {
    "self": {
      "href": "/users/101"
    },
    "orders": {
      "href": "/users/101/orders"
    }
  }
}
```

Do all REST APIs require HATEOAS?

No.

Many APIs commonly called REST APIs don't implement full HATEOAS.

Interview answer

> *HATEOAS is a REST constraint where representations contain hypermedia controls that help clients discover possible next actions. It is part of the REST architectural model, but many practical HTTP APIs use a more pragmatic REST style without full HATEOAS.*

PART C — VALIDATION, ERRORS, PAGINATION

Q31–Q45

Q31. How do you validate REST request data in Spring Boot? □

Use Bean Validation.

Example:

```java
public class CreateUserRequest {

    @NotBlank
    private String name;

    @Email
```

```
    @NotBlank
    private String email;

    @Min(18)
    private int age;
}
```

Controller:

```java
@PostMapping
public UserResponse create(
        @Valid @RequestBody CreateUserRequest request) {

    return userService.create(request);
}
```

Flow

```text
HTTP JSON
   ↓
Jackson
   ↓
DTO
   ↓
Bean Validation
   ↓
Controller
```

Q32. What is the difference between `@Valid` and `@Validated`? ⭐

`@Valid`

Comes from Jakarta Bean Validation and triggers validation.

```java
@Valid @RequestBody CreateUserRequest request
```

`@Validated`

Spring's validation annotation and supports validation groups and some method-level validation scenarios.

Example:

```java
@Validated
@Service
public class UserService {
}
```

Interview answer

> *@Valid is standard Bean Validation, while @Validated is Spring's variant that additionally supports validation groups and integrates with Spring's validation mechanisms.*

Q33. What happens when request validation fails?

Suppose:

```java
@NotBlank
private String name;
```

Request:

```json
{
  "name": ""
}
```

Validation fails before the controller method executes normally.

Spring may raise validation-related exceptions depending on where validation occurs.

API should return a structured error

Instead of:

```text
500 Internal Server Error
```

return something like:

```json
{
  "status": 400,
  "message": "Validation failed",
```

```
  "errors": {
    "name": "must not be blank"
  }
}
```

Q34. How do you implement global exception handling? □□

Use:

```java
@RestControllerAdvice
```

Example:

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ApiError> handle(
            UserNotFoundException ex) {

        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(new ApiError("USER_NOT_FOUND",
                                   ex.getMessage()));
    }
}
```

The previous material specifically established `@RestControllerAdvice` + `@ExceptionHandler` as the preferred centralized approach.

Q35. Why shouldn't every controller have try-catch?

Bad:

```java
try {
    ...
} catch (...) {
```

```
    ...
}
```

repeated across:

```
text
Controller A
Controller B
Controller C
Controller D
```

This causes:

- duplication
- inconsistent errors
- difficult maintenance

Better:

```
text
Controller
  ↓
Exception
  ↓
@RestControllerAdvice
  ↓
standard error response
```

Q36. How should a good REST error response be designed? □

Avoid returning random strings:

```
json
"something went wrong"
```

Use a consistent structure.

Example:

```
json
{
  "timestamp": "2026-08-23T10:00:00Z",
  "status": 404,
```

```
  "code": "USER_NOT_FOUND",
  "message": "User 101 was not found",
  "path": "/api/users/101"
}
```

For validation:

```
json
{
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "errors": [
    {
      "field": "email",
      "message": "must be a valid email"
    }
  ]
}
```

Important

Don't expose:

```
text
stack trace
SQL query
DB credentials
internal exception details
```

to clients.

Q37. What is RFC 7807 / Problem Details? ⭐

Modern APIs can use a standardized problem representation.

Conceptually:

```
json
{
  "type": "/problems/user-not-found",
  "title": "User Not Found",
```

```
  "status": 404,
  "detail": "User 101 does not exist",
  "instance": "/api/users/101"
}
```

Why useful?

It provides a standardized structure for HTTP API errors.

Interview answer

> *Problem Details gives APIs a standardized machine-readable error representation instead of inventing unrelated error structures for every endpoint.*

Q38. What is pagination and why is it needed? □□

Suppose:

```
text
10 million users
```

You shouldn't return all users in one response.

Pagination limits the amount of data returned.

Example:

```
http
GET /users?page=0&size=20
```

Response:

```
json
{
  "content": [...],
  "page": 0,
  "size": 20,
  "totalElements": 10000,
  "totalPages": 500
}
```

Benefits

- lower memory
- smaller response

- faster API
- reduced DB/network load

Q39. Offset pagination vs cursor/keyset pagination? □□

Offset pagination

```
http
 GET /users?page=100&size=20
```

SQL conceptually:

```
sql
 LIMIT 20 OFFSET 2000
```

Simple but can become inefficient for large offsets.

Keyset pagination

Instead of saying:

> *Give me page 100.*

you say:

> *Give me records after this last ID.*

Example:

```
http
 GET /users?afterId=2000&limit=20
```

Conceptually:

```
sql
 WHERE id > 2000
 ORDER BY id
 LIMIT 20
```

Product-company answer

> *Offset pagination is simple and useful for moderate datasets, while keyset pagination is generally better for large datasets or high-throughput feeds where deep offsets become expensive.*

Q40. Why can offset pagination become slow?

Consider:

```sql
 LIMIT 20 OFFSET 10_000_000
```

The database may need to scan/skip a large number of rows before returning the requested page.

Keyset approach

```sql
 WHERE id > :lastId
 ORDER BY id
 LIMIT 20
```

The database can leverage an appropriate index.

Important

Keyset pagination requires a stable ordering.

Q41. What are filtering and sorting in REST APIs?

Example:

```http
 GET /products?category=mobile&minPrice=10000
```

Filtering:

```text
 category
 minPrice
 maxPrice
```

Sorting:

```http
 GET /products?sort=price,asc
```

Design principle

Don't create dozens of endpoints:

```text
 /productsByPrice
```

```
/productsByCategory
/productsByCategoryAndPrice
```

Prefer query parameters where appropriate.

Q42. How would you design a search API?

Example:

```http
GET /products?
    search=iphone
    &category=mobile
    &minPrice=50000
    &maxPrice=150000
    &sort=price,asc
    &page=0
    &size=20
```

Backend flow

```text
Request
  ↓
DTO/query parameters
  ↓
Validation
  ↓
Specification/query builder
  ↓
Database
  ↓
Pagination
  ↓
DTO
  ↓
Response
```

Important

Don't blindly concatenate SQL strings.

Use parameterized queries / repository mechanisms.

Q43. What is a partial response?

Sometimes the client needs only a subset of fields.

Instead of:

```json
{
  "id": 1,
  "name": "...",
  "description": "...",
  "price": "...",
  "reviews": "...",
  "inventory": "..."
}
```

it may request:

```http
GET /products/10?fields=id,name,price
```

Why?

Reduced:

```text
DB work
serialization
network payload
client processing
```

Caveat

Field selection adds API complexity and shouldn't be introduced without a real need.

Q44. How would you design bulk APIs?

Suppose the client needs to create 1,000 users.

Instead of:

```text
POST /users
POST /users
POST /users
...
```

you may support:

```http
POST /users/bulk
```

with:

```json
{
  "users": [
    {...},
    {...},
    {...}
  ]
}
```

Questions interviewer may ask

- Is it atomic?
- What if user 500 fails?
- Should successful records remain?
- What response identifies failures?
- How large can the batch be?

Strong answer

> *I would explicitly define atomicity and partial-failure semantics rather than assuming that a bulk request behaves like a single database transaction.*

Q45. Synchronous vs asynchronous API? □□

Synchronous

Client waits for completion.

```text
Client
  ↓
```

```
POST /orders
 ↓
process
 ↓
201/200
```

Asynchronous

Server accepts the request and processing happens later.

```text
Client
 ↓
POST /reports
 ↓
202 Accepted
 ↓
Job created
 ↓
Background processing
```

Response:

```json
{
  "jobId": "abc123",
  "status": "PROCESSING"
}
```

Client may later call:

```http
GET /jobs/abc123
```

When useful?

Long-running operations:

```text
report generation
large exports
video processing
```

```
bulk processing
external integrations
```

PART D — IDEMPOTENCY, CACHING, CONCURRENCY, SECURITY

Q46–Q60

Q46. How would you make a POST API idempotent? □□□

Classic product-company question.

Suppose:

```
http
 POST /payments
 Idempotency-Key: payment-123
```

Request:

```
json
 {
   "amount": 1000,
   "currency": "INR"
 }
```

Server logic

```
text
 Receive request
  ↓
 Read idempotency key
  ↓
 Check stored key
  ↓
 Already processed?
  ├── YES → return previous result
  └── NO
        ↓
  process payment
        ↓
```

```
 store result against key
        ↓
 return response
```

Database design

Could maintain:

```
text
 idempotency_key
 request_hash
 status
 response
 created_at
```

with a unique constraint on:

```
text
 idempotency_key
```

Tricky question

What if two requests with the same key arrive simultaneously?

You need concurrency control.

For example:

```
text
 unique constraint
 transaction
 locking
 atomic insert
```

The unique key is often a strong protection against duplicate processing.

Q47. What is retry-safe API design?

Distributed systems experience:

```
text
 Request sent
   ↓
 Server processes
```

```
 ↓
Response lost
 ↓
Client thinks request failed
 ↓
Retry
```

Therefore API operations should be designed with retries in mind.

Example

GET is naturally retry-friendly.

PUT can be retry-friendly because of idempotency.

POST may require:

```text
Idempotency-Key
```

Interview answer

> *I don't treat retries as just a client concern. API semantics must be designed so that network retries don't accidentally duplicate business operations.*

Q48. What is optimistic concurrency control in REST APIs? □□

Suppose two users retrieve:

```text
User version = 5
```

Both modify the resource.

User A updates first:

```text
version 5 → 6
```

User B still has:

```text
version 5
```

If B blindly updates, it may overwrite A's changes.

Solution

Use version information.

For example:

```http
If-Match: "version-5"
```

Server verifies current version.

If it changed:

```text
412 Precondition Failed
```

Another possibility

Expose:

```json
{
  "id": 10,
  "version": 5
}
```

and require the version during update.

Q49. What is ETag? □□

ETag is an HTTP response header representing a version/fingerprint of a resource representation.

Example:

```http
ETag: "abc123"
```

Client later sends:

```http
If-None-Match: "abc123"
```

If resource hasn't changed:

```text
304 Not Modified
```

Benefits

Reduces unnecessary response body transfer.

Another use

`ETag` + `If-Match` can support optimistic concurrency.

Q50. `If-None-Match` vs `If-Match`?

`If-None-Match`

Often used for cache validation.

```
http
 If-None-Match: "abc123"
```

If unchanged:

```
text
 304 Not Modified
```

`If-Match`

Often used for concurrency control.

```
http
 If-Match: "abc123"
```

If current version doesn't match:

```
text
 412 Precondition Failed
```

Easy memory trick

```
text
 If-None-Match
 → cache validation


 If-Match
 → safe update/concurrency
```

Q51. What is HTTP caching?

Caching stores a response so it doesn't need to be regenerated or retrieved every time.

```text
Client
 ↓
Cache
 ↓
API
 ↓
Database
```

Potential benefits:

```text
lower latency
lower DB load
lower network cost
higher throughput
```

Important headers

```text
Cache-Control
ETag
Last-Modified
Expires
Vary
```

Q52. Explain `Cache-Control`.

Example:

```http
Cache-Control: max-age=3600
```

means the response can be considered fresh for a specified duration.

Other directives include:

```text
no-cache
no-store
private
public
```

```
max-age
must-revalidate
```

Important distinction

`no-cache` does **not** literally mean:

> *Don't store.*

It generally means cached content must be revalidated before reuse.

`no-store` means the response should not be stored.

This is a classic interviewer trap.

Q53. What is `304 Not Modified`?

Suppose:

```text
First request
  ↓
200 OK
ETag: "abc"
```

Client caches it.

Later:

```http
GET /products/10
If-None-Match: "abc"
```

Server checks the resource.

If unchanged:

```text
304 Not Modified
```

No response body is required.

Benefit

The client can reuse its cached representation.

Q54. What is rate limiting? □□

Rate limiting restricts how many requests a client can make during a period.

Example:

```
text
 100 requests/minute
```

If exceeded:

```
text
 429 Too Many Requests
```

Why?

Protect against:

```
text
 abuse
 DDoS-like traffic
 accidental overload
 misbehaving clients
 expensive API calls
```

Common algorithms

```
text
 Fixed Window
 Sliding Window
 Token Bucket
 Leaky Bucket
```

Q55. Explain Token Bucket rate limiting. ⭐

Imagine a bucket containing tokens.

```
text
 Bucket capacity = 100
 Refill = 10 tokens/sec
```

Each request consumes one token.

```
text
 Request
  ↓
 Token available?
  ├── YES → process
  └── NO → 429
```

Advantage

Allows controlled bursts while maintaining an average rate.

Q56. Where should rate limiting be implemented?

Possible locations:

```
text
 API Gateway
 Load Balancer
 Application
 Redis
 Distributed rate limiter
```

For distributed applications, a shared mechanism such as Redis is often useful.

Example

```
text
 Client
  ↓
 API Gateway
  ↓
 Rate Limiter
  ↓
 Service
```

Strong answer

> *For cross-service/global API protection I prefer enforcing limits close to the edge, while application-level limits may still be needed for business-specific operations.*

Q57. What is CORS? □

CORS = **Cross-Origin Resource Sharing**.

It controls whether browser-based applications can make requests to another origin.

Example:

```
text
 Frontend:
 https://app.example.com


 Backend:
 https://api.example.com
```

These are different origins.

Browser sends/request may involve:

```
http
 Origin: https://app.example.com
```

Server can respond with:

```
http
 Access-Control-Allow-Origin: https://app.example.com
```

Important

CORS is primarily a **browser security mechanism**.

It is not an authentication mechanism.

Q58. What is a preflight request?

For certain cross-origin requests, the browser sends:

```
http
 OPTIONS
```

before the actual request.

Example:

```
http
 OPTIONS /users
 Origin: https://app.example.com
 Access-Control-Request-Method: POST
```

Server responds with permitted methods/headers.

Then browser sends:

```http
POST /users
```

Interview trap

Does Postman usually enforce browser CORS restrictions?

No.

CORS is enforced by browsers.

Q59. How do you secure REST APIs? □□□

Use multiple layers.

```text
HTTPS
 ↓
Authentication
 ↓
Authorization
 ↓
Input validation
 ↓
Rate limiting
 ↓
Secure headers
 ↓
Logging/monitoring
```

Authentication

Examples:

```text
JWT
OAuth 2.0
session-based authentication
```

Authorization

```text
ROLE_USER
ROLE_ADMIN
permissions
resource ownership
```

Other concerns

- TLS/HTTPS
- input validation
- SQL injection prevention
- secure secrets
- CORS configuration
- CSRF depending on architecture
- rate limiting
- audit logging

Q60. Authentication vs Authorization? □□

Authentication

> *Who are you?*

```text
username/password
JWT
OAuth token
```

Authorization

> *What are you allowed to do?*

Example:

```text
Authenticated user
        ↓
ROLE_USER
        ↓
GET /orders
        ↓
Allowed
```

But:

```
text
 ROLE_USER
        ↓
 DELETE /admin/users
        ↓
 403
```

Memory trick

```
text
 Authentication = Identity
 Authorization = Permission
```

PART E — PRODUCTION SCENARIOS + ADVANCED INTERVIEW QUESTIONS

Q61–Q75

This is where the previous interview-bible philosophy becomes important.

The source material explicitly says that product-company interviews should move from basic questions into scenarios such as slow APIs, incorrect errors, huge query counts and production failures.

Q61. Your API suddenly becomes slow. How do you investigate? □□□

Don't immediately say:

> *Increase server resources.*

Use a systematic approach.

```
text
 Client
  ↓
 API latency
  ↓
 Application metrics
  ↓
 DB latency
  ↓
```

```
External calls
 ↓
Network
 ↓
Infrastructure
```

Step 1 — Measure

Check:

```
text
p50
p95
p99
error rate
throughput
CPU
memory
GC
```

Step 2 — Break latency down

```
text
Controller
 ↓
Service
 ↓
DB
 ↓
External API
```

Step 3 — Database

Check:

```
text
slow queries
N+1
missing indexes
execution plan
connection pool
locks
```

Step 4 — External dependencies

Check:

```
text
 timeout
 retry
 connection latency
 dependency availability
```

Step 5 — Application

Check:

```
text
 CPU
 GC
 thread pools
 connection pool
 serialization
 large payloads
```

Strong interview answer

> *I would first establish whether the latency increase is application-wide or endpoint-specific, then use metrics and tracing to isolate database, application, network and dependency latency rather than guessing at the root cause.*

Q62. Your API returns 10,000 DB queries for one request. What's happening? □□□

Immediately suspect:

```
text
 N+1
```

Especially with JPA/Hibernate.

Example:

```
text
 1 query → orders


 1000 orders
```

```
  ↓
 1000 customer queries
```

Total:

```
text
 1 + 1000 = 1001
```

The JPA module already established this production pattern and recommends examining SQL logs, Hibernate statistics, query count and execution time.

Solutions

Depending on use case:

```
text
 JOIN FETCH
 EntityGraph
 DTO projection
 batch fetching
 query redesign
```

Don't simply change everything to `EAGER`.

That previous module explicitly warns that blindly changing LAZY to EAGER can result in more SQL, larger object graphs and more memory consumption.

Q63. Your API receives duplicate payment requests. How do you solve it? □□□

This is an idempotency problem.

Use:

```
http
 Idempotency-Key: 7f8a...
```

Server

```
text
 Request
  ↓
 Idempotency key
  ↓
 Check DB/Redis
```

```
 ↓
Already processed?
 ├── yes → return stored result
 └── no
       ↓
 process
       ↓
 persist result
```

Critical issue

Two identical requests can arrive simultaneously.

Therefore the key must be protected using an atomic mechanism such as:

```
text
 unique DB constraint
 distributed lock
 atomic Redis operation
 transaction
```

Q64. Your client times out, but the server successfully processed the request. What happens when the client retries? □□□

This is one of the most important distributed API scenarios.

Timeline:

```
text
 Client
   |
   | POST
   ↓
 Server
   |
   | process
   ↓
 Database
   |
   | success
   ↓
```

```
Server
  X
  response lost
  |
Client timeout
  |
Retry
```

Without idempotency:

```text
Duplicate operation
```

With idempotency:

```text
same key
 ↓
previous result
 ↓
return previous result
```

Best answer

> *I design APIs assuming that network failures can happen after the server commits but before the client receives the response. For non-idempotent operations I use an idempotency key or another deduplication strategy.*

Q65. How do you handle downstream service timeout?

Suppose:

```text
Order API
   ↓
Payment API
```

Payment API doesn't respond.

Don't let the request hang forever.

Use:

```text
 connection timeout
 read timeout
 overall timeout
```

Then consider:

```text
 retry
 circuit breaker
 fallback
 async processing
```

Important

Retries should be controlled.

Bad:

```text
 retry forever
```

Good:

```text
 small retry count
 +
 backoff
 +
 jitter
 +
 timeout
```

Q66. Why is retrying every failed API call dangerous? □

Suppose downstream is overloaded.

100 requests arrive.

Each request retries 3 times.

Now downstream may receive:

```text
100 × 4 = 400 requests
```

This can make the outage worse.

This is sometimes called a retry storm.

Better

Use:

```text
timeouts
bounded retries
exponential backoff
jitter
circuit breaker
idempotency
```

Q67. What is exponential backoff?

Instead of:

```text
retry immediately
retry immediately
retry immediately
```

increase the delay:

```text
100 ms
200 ms
400 ms
800 ms
```

Usually with a maximum limit.

Add jitter

If thousands of clients retry at exactly:

```text
1 second
```

they can synchronize.

Jitter randomizes retry timing.

```
text
 800 ms + random jitter
```

Strong answer

> *Exponential backoff reduces pressure on a failing dependency, while jitter prevents many clients from retrying simultaneously.*

Q68. What is a circuit breaker? □□

A circuit breaker prevents continuous calls to an unhealthy dependency.

States:

```
text
 CLOSED
    ↓ failures
 OPEN
    ↓ after wait
 HALF_OPEN
    ↓ successful
 CLOSED
```

CLOSED

Requests flow normally.

OPEN

Calls fail fast.

```
text
 Request
  ↓
 Circuit OPEN
  ↓
 fallback/error
```

HALF_OPEN

Allow limited test requests.

If dependency recovers:

```
text
 HALF_OPEN → CLOSED
```

Otherwise:

```
text
 HALF_OPEN → OPEN
```

Why?

Prevent cascading failures.

Q69. What is an API gateway? □

An API Gateway is an entry point between clients and backend services.

```
text
                    ┌── User Service
 Client
    ↓
 API Gateway ──────┼── Order Service
                    ├── Payment Service
                    └── Product Service
```

Responsibilities can include

```
text
 routing
 authentication
 rate limiting
 TLS termination
 logging
 metrics
 request transformation
```

Important

Don't put all business logic into the gateway.

Business logic generally belongs in services.

Q70. How would you design a high-throughput GET API? □

Start with requirements.

Suppose:

```
text
 GET /products/{id}
```

receives:

```
text
 50,000 requests/sec
```

Possible architecture:

```
text
 Client
  ↓
 CDN / Cache
  ↓
 Load Balancer
  ↓
 Multiple API instances
  ↓
 Redis
  ↓
 Database
```

Optimize

1. HTTP caching
2. Redis/application cache
3. DB indexes
4. connection pooling
5. small payloads
6. pagination
7. compression where appropriate
8. horizontal scaling
9. observability

Interview trap

Don't say:

> *Add Redis.*

First ask:

> *Is the data cacheable? What is the consistency requirement? What is the cache invalidation strategy?*

Q71. What is the N+1 problem from a REST API perspective?

The API itself may look innocent:

```
http
 GET /orders
```

But internally:

```
text
 1 query → orders


 for each order:
     query customer
     query items
     query product
```

So:

```
text
 1 + N + N + N
```

The REST layer exposes the endpoint, but the actual problem may be JPA query behavior.

Product-company answer

> *REST latency problems aren't necessarily HTTP problems. I trace the complete request path into service, ORM, database and downstream dependencies.*

This connects directly to the Module 3 mental model.

Q72. How do you prevent over-fetching and under-fetching?

Over-fetching

Client receives more data than needed.

```
text
 GET /users/10
```

returns:

```text
 profile
 orders
 payments
 addresses
 preferences
 audit history
```

even though client needs only:

```text
 id
 name
```

Under-fetching

Client needs multiple API calls:

```text
 GET /user
 GET /user/orders
 GET /user/preferences
 GET /user/address
```

Solutions

Depending on the architecture:

```text
 well-designed resources
 specific endpoints
 field selection
 batch endpoints
 aggregation/BFF
 GraphQL in appropriate cases
```

Strong answer

> *I optimize for the client's actual use case while avoiding an API contract that becomes tightly coupled to every consumer.*

Q73. How would you design an API for a large file upload? □

Don't necessarily send a huge file through a normal synchronous application request.

Possible architecture:

```text
 Client
  ↓
 Upload initiation API
  ↓
 Pre-signed storage URL
  ↓
 Object storage
  ↓
 Upload
  ↓
 Event
  ↓
 Async processing
```

Example:

```http
 POST /files/uploads
```

Response:

```json
 {
   "uploadUrl": "...",
   "fileId": "abc123"
 }
```

Client uploads directly to object storage.

Benefits

```text
 less application-server bandwidth
 better scalability
 large-file support
 asynchronous processing
```

Q74. Design a production-grade `POST /orders` API. □□□

This is the type of question where the interviewer tests multiple concepts simultaneously.

Request

```http
POST /api/v1/orders
Content-Type: application/json
Idempotency-Key: order-123
Authorization: Bearer ...
```

Body:

```json
{
  "customerId": 101,
  "items": [
    {
      "productId": 10,
      "quantity": 2
    }
  ]
}
```

Step 1 — Authentication

```text
Bearer token
 ↓
authenticate
```

Step 2 — Authorization

Check:

```text
Can this user create an order?
```

Step 3 — Validation

```text
customerId required
items not empty
quantity > 0
```

Step 4 — Idempotency

```
text
 Idempotency-Key
  ↓
 already processed?
```

Step 5 — Business validation

```
text
 customer exists?
 product exists?
 product active?
 inventory available?
```

Step 6 — Transaction

Persist order and order items consistently.

Step 7 — Response

```
text
 201 Created
 Location: /api/v1/orders/5001
```

Response:

```
json
 {
   "id": 5001,
   "status": "CREATED"
 }
```

Important distributed-system issue

If payment is a separate service:

```
text
 Order DB transaction
        ↓
 Payment Service
```

Don't automatically keep one database transaction open across a slow remote call.

That belongs to distributed transaction/saga design, which we'll explore deeply in **Module 5 — Microservices**.

Q75. Final scenario: Design and explain a production-grade REST API end-to-end. □□□

This is the **master question for Module 4**.

Imagine:

```
text
 POST /api/v1/orders
```

Complete architecture

```
text
                          Client
                            |
                            | HTTPS
                            ↓
                     Load Balancer
                            |
                            ↓
                      API Gateway
                            |
                  ┌─────────┴─────────┐
                  | Authentication    |
                  | Rate Limiting     |
                  | Routing           |
                  └─────────┬─────────┘
                            |
                            ↓
                     Order Service
                            |
            ┌───────────────┼────────────────┐
            ↓               ↓                ↓
       Validation      Idempotency     Business Logic
            |               |                |
            └───────────────┼────────────────┘
                            ↓
                       Transaction
```

```
                          |
                          ↓
                      Database
                          |
                          ↓
                  Response DTO
                          |
                          ↓
              HttpMessageConverter
                          |
                          ↓
                      JSON/HTTP
                          |
                          ↓
                        Client
```

Request

```
http
 POST /api/v1/orders
 Authorization: Bearer <token>
 Content-Type: application/json
 Idempotency-Key: abc-123
```

Validation

```
text
 DTO validation
 +
 business validation
```

Persistence

```
text
 Service
  ↓
 @Transactional
  ↓
 Repository
  ↓
 JPA
```

```
  ↓
 Hibernate
  ↓
 SQL
  ↓
 Database
```

That connects directly to the persistence model established in Module 3. The previous JPA material's central chain was:

```text
 REST API
  ↓
 Service
  ↓
 @Transactional
  ↓
 Repository
  ↓
 Spring Data JPA
  ↓
 EntityManager
  ↓
 Persistence Context
  ↓
 Hibernate
  ↓
 JDBC
  ↓
 Database
```

Response

```http
 HTTP/1.1 201 Created
 Location: /api/v1/orders/5001
 Content-Type: application/json
```

```json
 {
   "id": 5001,
```

```
  "status": "CREATED"
}
```

Failure handling

```
text
Validation failure
        ↓
400

Authentication failure
        ↓
401

Authorization failure
        ↓
403

Resource missing
        ↓
404

Duplicate/conflicting state
        ↓
409

Rate limit exceeded
        ↓
429

Unexpected application failure
        ↓
500

Dependency unavailable
        ↓
503
```

If API becomes slow

Investigate:

```text
HTTP latency
 ↓
application metrics
 ↓
tracing
 ↓
DB queries
 ↓
N+1
 ↓
indexes
 ↓
connection pool
 ↓
external services
 ↓
CPU / memory / GC
```

If client retries

```text
Idempotency-Key
```

If two users update simultaneously

```text
ETag / If-Match
```

or application-level versioning.

If traffic increases

```text
Load balancing
+
horizontal scaling
+
caching
+
```

```
database optimization
+
rate limiting
```

If downstream service fails

```
text
Timeout
+
bounded retry
+
exponential backoff
+
jitter
+
circuit breaker
```

□ Module 4 — The 75 Questions You Should Be Able to Recall

REST fundamentals

1. What is REST?
2. REST principles?
3. What is a resource?
4. How should REST URLs be designed?
5. HTTP methods?
6. PUT vs PATCH?
7. POST vs PUT?
8. What is idempotency?
9. Which HTTP methods are idempotent?
10. What is statelessness?
11. Stateful vs stateless?
12. Safe vs idempotent?
13. HTTP status codes?
14. 401 vs 403?
15. 400 vs 422?

API design

1. Complete Spring REST request flow

2. `@RequestParam` vs `@PathVariable` vs `@RequestBody`
3. Content negotiation
4. `Content-Type`
5. `Accept`
6. DTOs
7. Entity vs DTO
8. API contract
9. Backward compatibility
10. API versioning
11. Should every API be versioned?
12. `ResponseEntity`
13. `ResponseEntity` vs object return
14. `Location` header
15. HATEOAS

Validation/errors/pagination

1. REST validation
2. `@Valid` vs `@Validated`
3. Validation failure flow
4. Global exception handling
5. Why centralized exception handling?
6. REST error structure
7. Problem Details
8. Pagination
9. Offset vs keyset
10. Deep-offset performance
11. Filtering/sorting
12. Search API
13. Partial response
14. Bulk APIs
15. Synchronous vs asynchronous API

Distributed/API concerns

1. POST idempotency
2. Retry-safe API
3. Optimistic concurrency

4. ETag
5. `If-Match` vs `If-None-Match`
6. HTTP caching
7. Cache-Control
8. 304 Not Modified
9. Rate limiting
10. Token Bucket
11. Where to implement rate limiting?
12. CORS
13. Preflight
14. REST security
15. Authentication vs authorization

Product-company scenarios

1. Slow API investigation
2. 10,000 DB queries/N+1
3. Duplicate payment requests
4. Timeout + retry duplicate
5. Downstream timeout
6. Dangerous retries
7. Exponential backoff + jitter
8. Circuit breaker
9. API Gateway
10. High-throughput API
11. REST + N+1
12. Over-fetching/under-fetching
13. Large file upload
14. Production `POST /orders`
15. Complete production REST API design

□ What I want you to be able to do after Module 4

Don't memorize 75 definitions.

You should be able to take an interviewer question such as:

> *"Design a REST API for creating an order."*

and naturally walk through:

```text
Resource modeling
      ↓
URI
      ↓
HTTP method
      ↓
DTO
      ↓
Validation
      ↓
Authentication
      ↓
Authorization
      ↓
Idempotency
      ↓
Service
      ↓
Transaction
      ↓
Repository
      ↓
Database
      ↓
DTO mapping
      ↓
HTTP status
      ↓
Error handling
      ↓
Observability
      ↓
Caching / performance
      ↓
Retry / timeout
```

```
      ↓
Concurrency
```

That is the **same interview progression established in Modules 1–3**: start with the simple definition, then go deeper until the interviewer reaches internal behavior and finally a production scenario. The earlier material explicitly describes this progression as the difference between basic preparation and product-company-ready thinking.

And Module 4 now connects cleanly to the previous modules:

```text
MODULE 1
Core Java
   ↓
MODULE 2
Spring / Spring Boot
   ↓
MODULE 3
JPA / Hibernate
   ↓
MODULE 4
REST / API
   ↓
MODULE 5
Microservices
```

Next in the sequence is Module 5 — 75 Microservices questions, where these REST concepts become distributed-system concepts: **service-to-service communication, service discovery, API Gateway, load balancing, circuit breakers, retries, timeouts, resilience, distributed transactions/Saga, Kafka, eventual consistency, observability, Docker/Kubernetes basics, and real production scenarios.**